

CHAPTER 9

Managing Real-Time Streaming Ingestion with Azure Event Hub

Introduction

Organizations increasingly depend on real-time data to detect operational issues, respond to customer behavior, and power time-sensitive analytics. From application telemetry and **Internet of Things (IoT)** signals to user interactions and system events, data is now generated continuously and at high velocity. However, ingesting, processing, and managing these event streams reliably at scale introduces new challenges, including throughput management, ordering guarantees, fault tolerance, cost control, and downstream integration.

Azure Event Hub was designed to address these challenges. It is a fully managed, high-throughput event ingestion service capable of receiving millions of events per second from diverse producers. Acting as a central streaming backbone, Event Hub decouples event producers from consumers, enabling multiple downstream systems to process the same data independently without duplication or tight coupling.

This chapter serves as a practical guide to designing and implementing real-time streaming ingestion using Azure Event Hub. Rather than treating streaming as an isolated capability, the chapter positions Event Hub within modern data architectures where real-time ingestion complements batch processing, Lakehouse storage, and analytical workloads. You will learn how Event Hub enables scalable ingestion while integrating seamlessly with downstream services such as Azure Data Lake, Spark-based processing engines, and real-time analytics platforms.

The chapter begins with the fundamentals of Azure Event Hub architecture and core concepts, then moves through hands-on setup, event publishing patterns, data capture, and stream processing approaches. Along the way, it addresses operational concerns such as

monitoring, scaling, security, and troubleshooting, ensuring that real-time pipelines remain reliable and cost-effective in production environments.

By the end of this chapter, you will have a clear understanding of how to design, build, and operate real-time ingestion pipelines using Azure Event Hub as a foundational streaming service within modern Azure data platforms.

Structure

This chapter covers the following topics:

- Real-time streaming ingestion
- Azure Event Hub fundamentals and architecture
- Creating and configuring Event Hubs for scalable ingestion
- Publishing events and designing streaming data schemas
- Capturing and persisting streaming data
- Processing real-time data
- Integrating with other services
- Monitoring, scaling, and reliability
- Securing Event Hub access and managing permissions
- Troubleshooting and operational best practices

Objectives

The objective of this chapter is to provide a clear and practical understanding of Azure Event Hub as a foundational service for real-time data ingestion in modern data engineering architectures. This chapter explains how Event Hub enables organizations to ingest high-volume, high-velocity event streams reliably while decoupling data producers from downstream consumers.

Readers will learn how Azure Event Hub simplifies real-time ingestion by offering built-in scalability and partitioning without requiring custom infrastructure management. By exploring key concepts such as namespaces, partitions, consumer groups, throughput units, and capture, the chapter demonstrates how to design streaming ingestion pipelines that are both performant and cost-aware.

This chapter also illustrates how Azure Event Hub integrates with downstream processing and analytics services, including Spark-based streaming engines, data lakes, and real-time analytics platforms.

By the end of this chapter, readers will gain the confidence to design, implement, and operate robust real-time ingestion pipelines using Azure Event Hub, enabling timely insights while maintaining operational simplicity and governance.

Real-time streaming ingestion

Traditional data platforms were designed around batch processing, in which data is collected over a period, stored, and processed in scheduled intervals. While this model remains effective for historical analysis and reporting, it is increasingly insufficient for use cases that demand timely insights. Modern digital systems continuously generate data through user interactions, application logs, device telemetry, transactions, and operational signals, all of which lose value if processed hours or days later.

Real-time streaming ingestion addresses this challenge by enabling data to be captured and processed as events occur. Instead of waiting for files to land or batches to complete, streaming systems continuously ingest events, allowing downstream consumers to react with minimal latency. This shift is critical for operational monitoring, fraud detection, near real-time dashboards, alerting, personalization, and event-driven automation.

However, building streaming ingestion pipelines introduces complexities that are not typically encountered in batch systems. Streaming data arrives at high velocity and in unpredictable volumes. Events may arrive out of order, be duplicated, or experience temporary spikes that overwhelm downstream systems. Additionally, multiple consumers often need access to the same stream for different purposes, such as real-time analytics, long-term storage, and operational workflows, without tightly coupling producers to consumers.

A robust real-time ingestion layer must therefore provide the following capabilities:

- High-throughput and low-latency ingestion to handle continuous event streams.
- Durable buffering to absorb spikes and protect downstream systems.
- Scalable partitioning to support parallel processing.
- Decoupling of producers and consumers to enable independent evolution.
- Re-playability to support reprocessing and recovery.
- Cost-efficient operation at scale.

Azure Event Hub fulfills this role within the Azure ecosystem. It acts as a distributed event ingestion service that sits between event producers, such as applications, services, and devices, and downstream processing or analytics systems. By providing a managed, scalable ingestion backbone, Event Hub allows data engineers to focus on data flow design and analytics logic rather than infrastructure management.

It is important to note that real-time ingestion does not replace batch processing. In most enterprise architectures, streaming and batch coexist. Streaming pipelines handle time-sensitive data, while batch pipelines continue to process large historical datasets, reference data, and backfills. Azure Event Hub fits naturally into this hybrid model by enabling real-time ingestion while integrating seamlessly with storage systems, Lakehouses, and analytics engines that also support batch workloads.

The following sections will explore how Azure Event Hub enables reliable real-time ingestion, starting with its core architecture and concepts. This foundation will help you make informed design decisions as we move into configuration, event publishing, capture, processing, and operational best practices.

Azure Event Hub fundamentals and architecture

To use Event Hub effectively, you need a handful of core concepts, how they relate to scale, and what trade-offs they embed. This section gives you the foundation to make sensible design choices before any hands-on work.

Namespace and Event Hub

A namespace is the container for one or more Event Hubs. It can be thought of as the top-level grouping, similar to a database or topic namespace in other messaging systems.

An Event Hub is the actual stream endpoint where producers publish events and consumers read them. You will configure each event hub's properties, such as partitions, retention, and access, when you create it.

Partitions

Events in an Event Hub are logically grouped into partitions, with each partition acting as an independent commit log that preserves event order. Partitions enable horizontal scaling by increasing **Input/Output (I/O)** throughput and allowing multiple consumers to process data in parallel, while ensuring each event has a single, well-defined owner.

 The number of partitions is set when the hub is created. For most tiers, other than premium or dedicated, you cannot reduce partitions later. Therefore, it is advisable to plan for peak load.

Partitions also determine event ordering. Events that share the same partition key are stored together and delivered in strict arrival order, providing ordering guarantees within a logical group.

Throughput units or processing units

Throughput Units (TUs) and **Processing Units (PUs)** are the capacity *ingredients* that determine how much data your Event Hub can handle at a time. TUs are used in the **Standard tier**, while PUs are used for the **Premium tier**, offering more dedicated resources and predictable performance. The key characteristics of TUs and Pus are as follows:

- In the Standard tier, one TU lets you ingest up to 1 MB/s (or 1,000 events per second) and deliver up to 2 MB/s (or 4,096 events per second). If you go beyond this, Event Hubs throttles the traffic; no data is lost, but ingestion slows down.
- TUs are billed hourly and shared across all Event Hubs in a namespace (up to 40). Enable Auto-Inflate to automatically scale TUs when traffic spikes.
- In the Premium tier, PUs provide dedicated throughput and stronger isolation. Microsoft recommends sizing PUs based on your expected ingestion rates, number of consumers, and latency needs.

Consumer groups and offset management

Consumer groups enable multiple downstream applications to read the same event stream independently, each at its own pace. They are the publish/subscribe mechanism in Event Hubs. Readers use offsets to track positions within partitions, and their progress is checkpointed to resume progress after failures. This is critical for reliability and replay.

Retention and storage

Events are retained for a configurable time window, with a default minimum of one hour and upper limits determined by the Event Hubs tier. Once this retention window expires, events are permanently deleted and cannot be replayed. Event Hubs is therefore not a long-term storage system but a transient ingestion buffer designed for real-time processing.

Workloads that require replay beyond the retention window, historical reprocessing, or regulatory archiving must explicitly persist events using Event Hubs Capture or forward them to external storage such as Azure Data Lake Storage, Blob Storage, or downstream

analytical systems. Retention should be sized only to absorb consumer lag and short operational outages, not to serve as a durable data store.

Key design considerations

The key design considerations are as follows:

- Decide partition count early based on expected parallel processing and ordering needs.
- Size throughput or processing units to handle peak load, keeping cost in check.
- Use consumer groups to support multiple downstream apps or storage writers, and plan checkpointing.
- Understand retention limits so you know when to rely on capture or downstream storage.

With these fundamentals, you are ready to configure and deploy a hub that can support real-time ingestion at scale, laying the groundwork for the hands-on setup and exercises that follow.

Azure Event Hubs reference architecture

A scalable real-time streaming platform on Azure Event Hubs can integrate diverse event sources into a centralized ingestion layer, process events using multiple Azure services, and model the resulting data to support analytics, applications, and business reporting. *Figure 9.1* illustrates this reference architecture, showing how these layers come together to form a cohesive streaming pipeline that supports both real-time responsiveness and downstream analytical use cases.

Event sources

At the foundation of the architecture are event sources that continuously generate data. These sources include streaming data such as user interactions and telemetry, application logs, file and media events, external feeds like weather and sentiment data, and domain events emitted by business applications. Each source operates independently and produces events at varying rates, making it impractical to connect them directly to analytical or storage systems.

This diversity and unpredictability in data generation necessitate a robust and scalable ingestion layer.

The following figure illustrates the reference architecture of Azure Event Hubs:

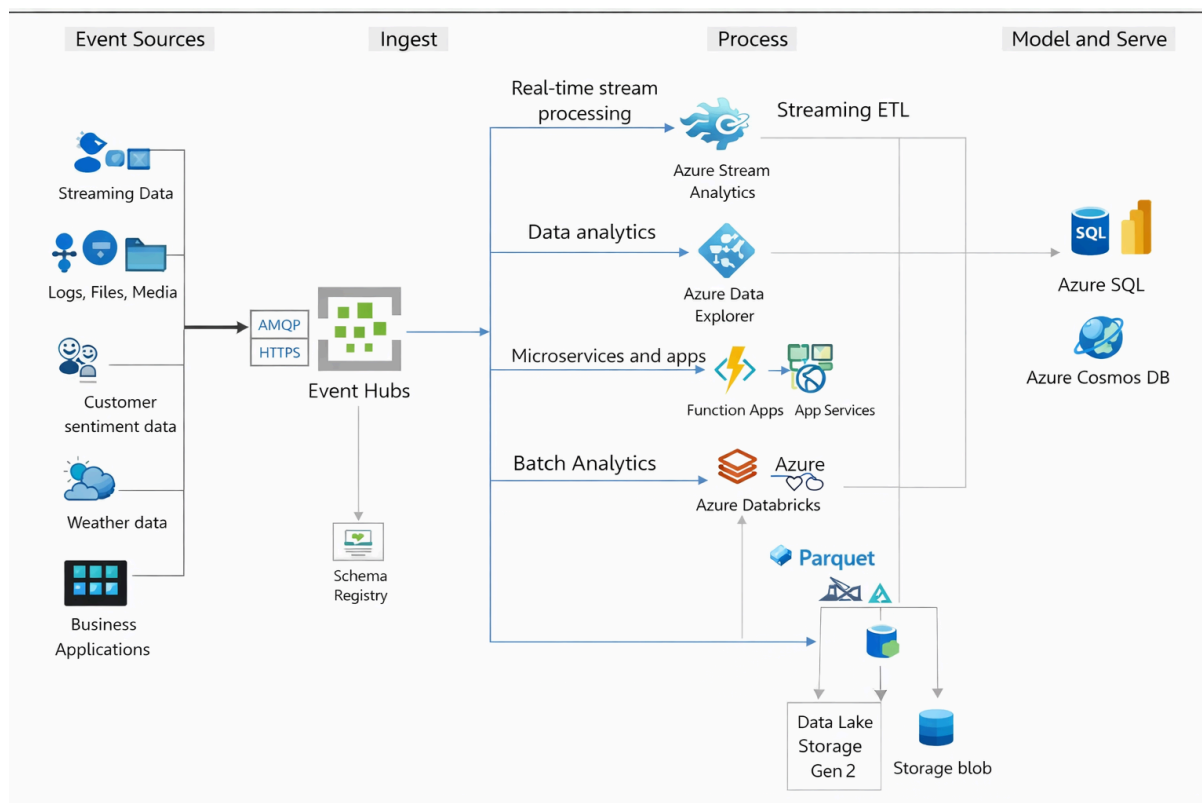


Figure 9.1: Azure Event Hubs reference architecture (Source: Microsoft)

Ingestion layer

The ingestion layer is responsible for receiving events reliably and making them available for downstream processing. Azure Event Hubs sits at the center of this layer, acting as a durable buffer between producers and consumers.

Events are published to Event Hubs using standard protocols such as **Advanced Message Queuing Protocol (AMQP)** or **Hypertext Transfer Protocol Secure (HTTPS)**, allowing a wide range of producers to send data securely and efficiently. Event Hubs absorbs traffic spikes, decouples producers from consumers, and ensures the stability of ingestion even under high load.

For structured event payloads, Schema Registry complements Event Hubs by enabling producers and consumers to share and validate schemas. This ensures consistency across event streams and allows schemas to evolve safely over time without disrupting downstream processing.

Processing layer

Once events are ingested, they flow into the processing layer, where multiple Azure services can independently consume and process the same event stream. This is made possible

through consumer groups, which allow each processing engine to maintain its own read position without interfering with others.

Common processing options include Azure Stream Analytics for low-latency, **Structured Query Language (SQL)**-based transformations, Azure Data Explorer for high-performance analysis of log and telemetry data, Azure Functions for event-driven processing, and application services for custom streaming workloads. Azure Databricks provides advanced stream processing using Spark Structured Streaming, supporting complex transformations, stateful processing, and integration with Lakehouse architectures.

This layer enables organizations to choose the most appropriate processing engine based on latency, complexity, and analytical requirements.

Model and serve layer

Processed streaming data is ultimately delivered to the model and serve layer, where it is shaped into formats suitable for consumption by applications and business users.

Azure Cosmos DB is commonly used to store processed or aggregated streaming data of applications with low-latency access. Azure SQL supports relational modeling and structured access patterns for downstream systems that rely on tabular representations. Power BI connects to these curated datasets to deliver real-time or near-real-time dashboards, enabling business users to monitor trends and operational metrics as events occur.

Creating and configuring Event Hubs

This recipe starts with setting up Azure Event Hub by creating a Namespace and an event hub and defining partitions and access policies as explained in this section.

Setting up Azure Event Hub

Before events can flow, the Event Hubs infrastructure must be ready. This setup defines the namespace, creates Event Hubs, configures partitions for scalability, selects cost-conscious pricing, and sets access policies. A clean setup ensures a reliable and secure ingestion layer.

Creating a namespace and Event Hub

The following sections explore the creation of an Azure Event Hubs namespace.

Objective: Create an Azure Event Hubs namespace and an Event Hub to act as the entry point for streaming data ingestion, enabling reliable intake of real-time events for downstream processing.

The following are the steps involved:

1. Sign in to the **Azure Portal**.
2. Search for **Event Hubs** and select the service.
3. Click **Create**.
4. Select your **Subscription** and **Resource Group**.
5. Enter a **Namespace name** (e.g., rt-ingestion-ns).
6. Select the **Region** closest to your event producers.
7. Choose a **Pricing tier** (Basic or Standard).
8. Click **Review + Create**.
9. After the namespace is deployed, open it and select **Event Hubs**.
10. Click **+ Event Hub**, enter a name (e.g., hc-events-eh), configure **Message retention**, and click **Review + Create**.

Outcome: An Event Hubs namespace and a single Event Hub are created and configured, ready to receive streaming events for downstream consumption.

The following figure shows the **Create Namespace** home screen:

Microsoft Azure

Search resources, services, and docs (G+)

Home > Event Hubs >

Create Namespace

Event Hubs

manage all your resources.

Subscription * Azure-DataEngineering

Resource group * rg-azde-dev-ci

[Create new](#)

Instance Details

Enter required settings for this namespace, including a price tier and configuring the number of units (capacity).

Namespace name * rt-ingestion-ns ✓
 .servicebus.windows.net

Region * Central India ✓
i The region selected supports Availability zones. Your namespace will have Availability Zones enabled. [Learn more.](#)

Pricing tier * Basic ✓
[Browse the available plans and their features](#)

Throughput Units * 1

[Review + create](#) [< Previous](#) [Next: Advanced >](#)

Figure 9.2: Create namespace home screen

Figure 9.3 shows the screen after creating the namespace. It is as follows:

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home >

rt-ingestion-ns | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ **Your deployment is complete**

Deployment name : rt-ingestion-ns Start time : 1/16/2026, 7:45:56 AM

Subscription : Azure-DataEngineering Correlation ID : 081961d8-4bed-40fc-97de-6f769155f7e2

Resource group : rg-azde-dev-ci

✓ **Deployment details**

Resource	Type	Status	Operation details
✓ rt-ingestion-ns	Event Hubs Namespace	OK	Operation details

✓ **Next steps**

[Go to resource](#)

Figure 9.3: rt-ingestion-ns namespace created

Figure 9.4 shows the **Create Event Hub** home screen; it is as follows:

The screenshot displays the 'Create Event Hub' page in the Microsoft Azure portal. The breadcrumb trail is 'Home > Event Hubs > rt-ingestion-ns >'. The page title is 'Create Event Hub' with a three-dot menu icon. Below the title, the 'Event Hubs' service is indicated. The 'Basics' tab is active, with 'Capture' and 'Review + create' tabs also visible. The 'Event Hub Details' section prompts the user to 'Enter required settings for this event hub, including partition count and message retention.' It contains a 'Name' field with the value 'hc-events-eh' and a green checkmark, and a 'Partition count' slider set to 2. The 'Retention' section prompts the user to 'Configure retention settings for this Event Hub. [Learn more](#)'. It includes a 'Cleanup policy' dropdown set to 'Delete' and a 'Retention time (hrs)' input field set to 1, with a note 'min. 1 hour, max. 24 hours (1day)'. At the bottom, there are three buttons: 'Review + create' (highlighted in blue), '< Previous', and 'Next: Capture >'.

Figure 9.4: Create Event Hub home screen

Configuring partitions and access policies

The following section explores the configuration of partitions and access policies.

Objective: Configure Event Hub partitions and access policies to enable efficient scaling and secure access for producers and consumers. This ensures reliable event flow while properly controlling the permissions.

The following are the steps to configure partitions:

1. While creating an Event Hub, locate the **partition count**.
2. Enter a number based on expected throughput as follows:
 - a. Two to four partitions for development

- b. More for production workloads
3. Confirm the configuration.

Refer to *Figure 9.4*. Partition count is provided as 2.

The following are the steps to configure shared access policies:

1. Navigate to the Event Hub (hc-events-eh).
2. In the left-hand navigation pane, scroll to the **Settings** section.
3. Select **Shared access policies**.
4. Click + **Add**.
5. Give policy name.

When creating or editing a policy, you can assign the following permissions:

- **Send**: Allows applications to publish events
- **Listen**: Allows applications to consume events
- **Manage**: Full control, including policy management

Best practice is to avoid using the **Manage** permission for applications. It is to be reserved for administrative access only.

6. Create a **Send** policy for producers (e.g., hc-producer-send-policy). This will be assigned to applications publishing healthcare events.
7. Create a **Listen** policy for consumers (e.g., hc-consumer-listen-policy). This will be assigned to Spark, Stream Analytics, or other consumers.
8. (Optional) Assign **Manage** only to administrators.
9. Copy the connection strings for use in applications.

Outcome: The Event Hub is partitioned for scalable consumption, and access policies secure the environment.

Figure 9.5 shows the Shared access policies screen. It is as follows:

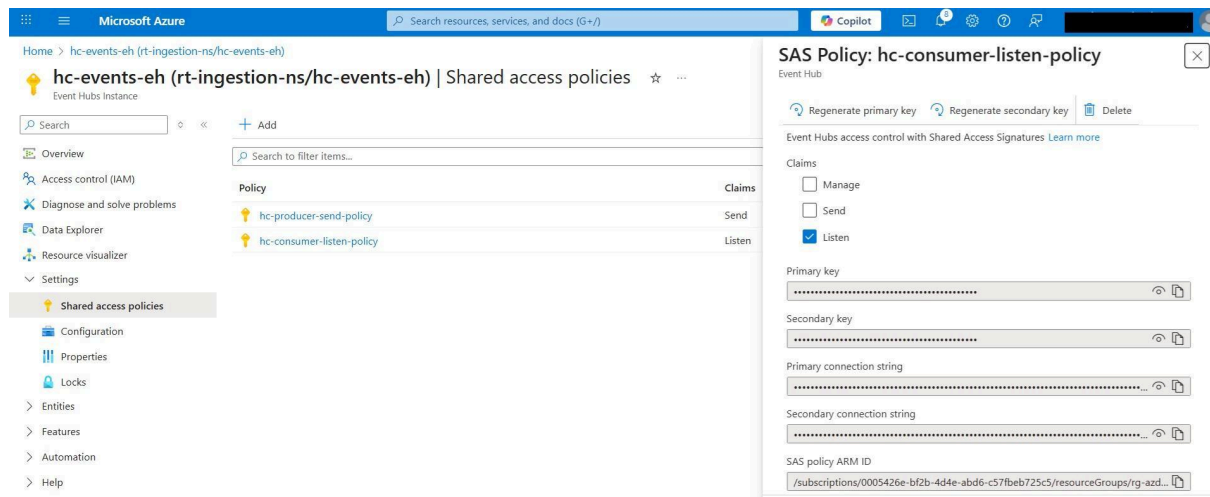


Figure 9.5: Shared access policies screen

By completing this exercise, the foundation for real-time ingestion is established. It is as follows:

- The namespace defines the environment and billing scope.
- Event Hubs represent streams ready to receive data.
- Partitions enable parallelism for high throughput.
- Shared access policies secure producer and consumer access.

The next step is to send events to Event Hub. It will demonstrate how producers can push data into this ingestion layer.

Publishing events and designing streaming data schemas

Now, let us cook the recipe for publishing events and designing data schemas in Azure Event Hub to send events and act on them, like enriching the data in store or creating notification alerts in Azure Monitor by following the steps provided in this section.

Sending events to Azure Event Hub

With the ingestion infrastructure in place, the next step is to activate the pipeline by publishing events into Azure Event Hubs. This marks the transition from configuration to real-time data flow and validates that the ingestion layer is functioning as designed.

In a streaming architecture, applications that publish data are called producers. Producers can be business applications, log emitting services, devices sending telemetry, or external

systems pushing events over HTTPS. Regardless of the source, the producer's responsibility is to package events correctly and deliver them to Event Hubs reliably.

This section focuses on the producer side of the architecture. Before introducing stream processing engines or analytics services, it is important to confirm that events are successfully reaching Event Hubs and that they are stored durably for consumption.

Understanding event producers

An event producer is any application or service that sends data to Azure Event Hubs. Producers do not need to be aware of downstream consumers or processing logic. One of the core strengths of Event Hubs is this decoupling. It allows ingestion to scale independently.

Producers communicate with Event Hubs using secure connection strings and standard protocols. Each event is typically represented as a lightweight message containing structured data. It is formatted as **JavaScript Object Notation (JSON)**. These events are appended to partitions within the Event Hub and made available to consumers based on retention settings.

At this stage, the goal is not to optimize producer throughput but to establish a reliable and verifiable flow of events into the platform.

Designing a simple event payload

It helps to define a simple and consistent payload structure before sending events. For learning and validation purposes, a minimal JSON payload is sufficient.

A typical event payload may include the following:

- An event identifier.
- A timestamp indicating when the event occurred.
- A business or system attribute, such as type, source, or value.

Keeping the payload small and readable makes it easier to verify ingestion and troubleshoot issues early.

Schema enforcement and evolution are not mandatory at this point. The focus is on ensuring that producers can successfully publish events and that Event Hubs receive them as expected.

Verifying event ingestion

Sending events without verification can lead to false confidence. It is important to confirm that events are arriving in Event Hubs and being retained correctly before introducing processors.

Azure provides basic tooling to observe incoming events and validate ingestion. This verification step ensures the following:

- Connection strings and access policies are correct.
- Events are reaching the intended Event Hub.
- Partitions are receiving data.

Once ingestion is validated, the platform is ready for downstream processing and analytics.

Sending events to Azure Event Hub

The following section explores the sending of events to Azure Event Hub.

Objective: Publish sample events to Azure Event Hub and verify successful ingestion.

The following are the steps involved in sending the events:

1. Open the **Azure Portal** and navigate to your **Event Hubs Namespace**.
2. Select the **Event Hub** created in the previous section (i.e., hc-events-eh).
3. From the left navigation, select **Data Explorer**.
4. Click the **Send events**.
5. Select **Custom payload** from the **Select Dataset** dropdown.
6. Select **JSON** from the **Select Content-Type** dropdown.
7. Browse and upload **hc-events.json**.

The following is a sample event payload:

```
{
  "event_id": "EVT-008",
  "event_type": "CONSULTATION",
  "event_time": "2023-04-06T12:05:00Z",
  "source_system": "ehr-system",
  "patient_id": "P008",
  "encounter_id": "E0008",
  "provider_id": "DR007",
  "department": "Dermatology",
  "encounter_type": "OPD",
  "estimated_cost": 1200,
```

```
"severity_level": "LOW"
}
```

8. **Send** a small batch of JSON events from the selected file.
9. Click **View events** and monitor incoming events.

Outcome: Events are successfully published to Azure Event Hub and are visible within the namespace. The ingestion layer is now active and ready for real-time processing by downstream services.

Figure 9.6 shows the selection made in the **Send events** screen. It is as follows:

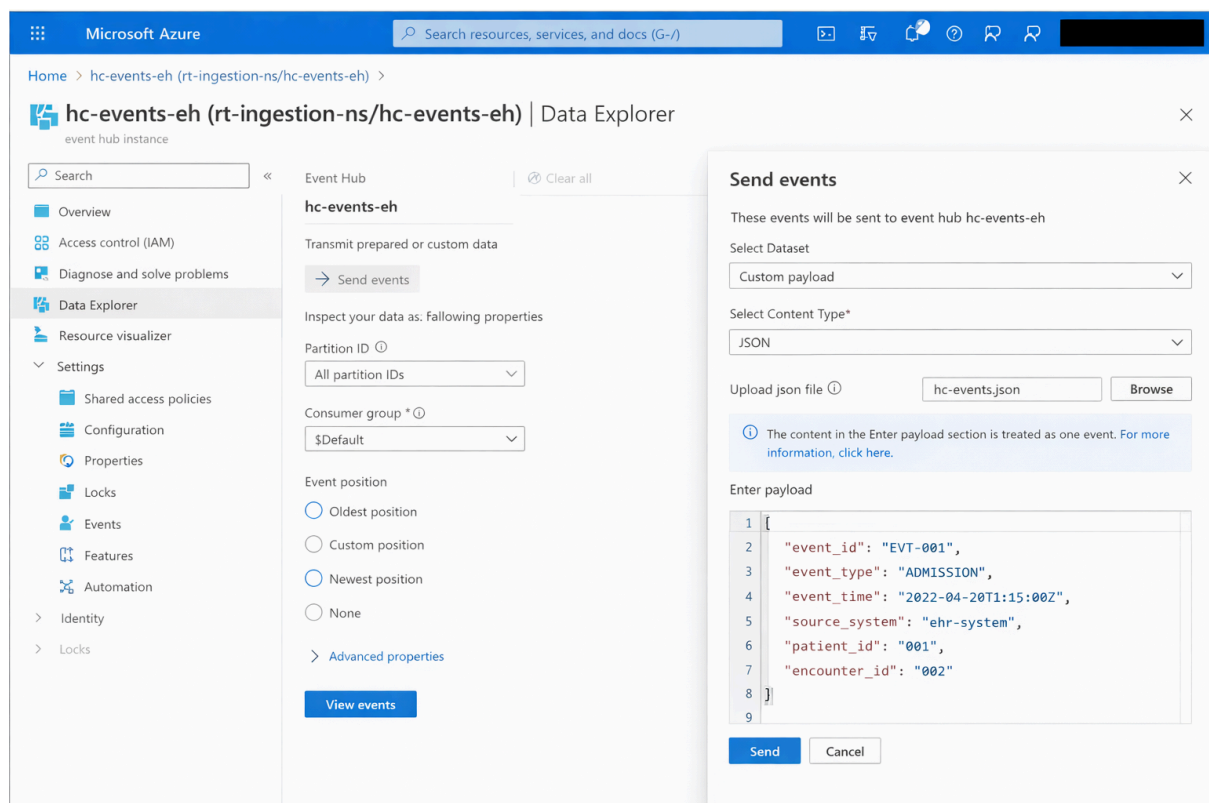


Figure 9.6: Selection in the send events screen

Understanding real-time behavior in the exercise

In the preceding exercise, the healthcare events were sent to Azure Event Hubs as a single JSON batch containing 15 events. This approach is intentional and keeps the hands-on walkthrough simple, repeatable, and cost-effective for learning purposes.

Although the events were submitted together, Azure Event Hubs ingests and stores each event individually. Once inside the Event Hub, every event receives its own enqueue time,

partition assignment, offset, and sequence number. From the platform's perspective, these are independent streaming events, not a single batch record.

When viewing the events using the View events experience, readers may notice that events appear almost instantly rather than gradually. This is expected because an ingestion occurred in an operation. However, the underlying architecture and processing model remain the same as a real-time system.

Common mistakes when sending events

The common mistakes when sending events to Azure Event Hub are as follows:

- **Using the Manage policy for producers:** Granting full permissions is unnecessary and risky. Producers should use **Send-only** access.
- **Sending oversized event payloads:** Large messages reduce throughput and increase latency. Event Hubs is optimized for many small events, not large blobs of data.
- **Ignoring partition strategy altogether:** While automatic partitioning works, completely ignoring partition behavior can cause uneven load in real-world systems.
- **Skipping ingestion verification:** Assuming events are flowing without validating metrics or ingestion views often leads to silent failures later.

Capturing and persisting streaming data

Real-time streaming systems are not only about speed. They must also support durability, replayability, and historical analysis. While Event Hubs retains events for a limited time, production-grade architectures almost always persist streaming data to long-term storage.

Azure Event Hub Capture addresses this requirement directly. Instead of building custom consumers to write events to storage, Capture provides a managed, cost-effective mechanism to automatically persist streaming data with minimal configuration.

This section explains how Event Hub Capture fits into the reference architecture and why it is the preferred approach for persisting streaming data at scale.

Persist streaming data

Streaming data is often processed in near real time, but business and analytical needs extend beyond immediate consumption. Persisting events enable the following:

- Reprocessing historical data with new logic.
- Backfilling downstream systems.
- Audit and compliance reporting.

- Machine learning and batch analytics.

Without persistence, streaming systems become fragile and tightly coupled to real-time consumers. Capture introduces durability without sacrificing simplicity.

Managed persistence by design in Event Hub Capture

Event Hub Capture is a built-in capability that automatically writes incoming events to Azure storage. It operates independently of consumers and does not require custom code or continuous compute resources.

From an architectural standpoint, Capture acts as a storage sink directly attached to the ingestion layer. Events are buffered internally and periodically written to storage in optimized batches.

This approach is significantly cheaper than custom consumers because of the following reasons:

- No always-on compute is required.
- No offset management or checkpointing logic is needed.
- Storage writes are optimized and batched.

Capture is therefore ideal for scenarios where raw or lightly processed streaming data must be retained.

Choosing storage

Event Hub Capture supports both Azure Blob Storage and **Azure Data Lake Storage (ADLS) Gen2**. The choice depends on downstream usage patterns. The two storage options have the following features:

- **Azure Blob Storage:** Suitable for simple archival, replay, or cost-sensitive retention scenarios.
- **Azure Data Lake Storage Gen2:** Preferred when captured data will be queried by analytics engines such as Azure Databricks, Synapse, or Fabric Lakehouse.

In modern analytics architectures, ADLS Gen2 is commonly chosen because it integrates naturally with big data and Lakehouse workloads.

File format and partitioned folder structure

Captured events are written in Avro format. It is a compact, schema-aware binary format optimized for high-throughput systems. Avro balances storage efficiency with fast read performance and is widely supported across Azure analytics services.

Event Hub Capture automatically organizes files using a time and partition-based folder structure, typically including the following:

- Event Hub name.
- Partition **Identifier (ID)**.
- Date and time hierarchy.

This structure enables parallel reads and efficient filtering when data is consumed later by Spark, SQL engines, or batch pipelines.

Common pitfalls with Event Hub Capture

Although Capture is simple to enable, the following mistakes can reduce its effectiveness:

- **Treating Capture as a real-time processing engine:** Capture is designed for persistence, not low-latency processing.
- **Ignoring file size and time window settings:** Poor configuration can result in too many small files or delayed writes.
- **Assuming Capture replaces all consumers:** Capture complements real-time consumers; it does not replace them.

Understanding these limitations ensures Capture is used properly in the architecture.

Capturing streaming data using Event Hub Capture

This exercise replaces the earlier standalone capture exercise and aligns with the unified ingestion-to-storage flow. The following section explores the exercise.

Objective: Enable Event Hub Capture to persist streaming data to Azure storage for long-term retention and analytics.

The following are the steps involved in the exercise:

1. Open the **Azure Portal** and navigate to your **Event Hubs Namespace**.
2. Select the **Event Hub** receiving streaming events.
3. From the left navigation, select **Features | Capture**.
4. Select **Avro** as the file format.
5. **Enable Capture**. (Standard tier needs to be selected at the Namespace level for this).



6. Select the Azure storage container for **streaming**.
7. Configure capture window settings (time or size).
8. Click **Apply** and allow events to flow.

Outcome: Streaming events are automatically persisted to Azure storage in a structured, partitioned format. The platform now supports replay, batch analytics, and long-term retention without custom ingestion code.

Figure 9.7 shows the Event Hub Capture configuration screen. It is as follows:

The screenshot displays the Azure portal's configuration page for Event Hub Capture. On the left, a navigation pane lists various services, with 'Capture' selected under the 'Features' section. The main area shows the configuration for 'hc-events-eh (rt-ingestion-ns/hc-events-eh) | Capture'. A descriptive text states that Event Hub Capture delivers streaming data to an Azure Blob storage account. A note mentions that enabling capture results in additional charges. The 'Enable capture' checkbox is checked. Below this, several settings are configured: 'Azure storage container' is 'streaming', 'Storage account' is a subscription path, 'Sample Capture file name formats' and 'Capture file name format' are both set to a structured path template, 'Time window (minutes)' is 1, and 'Size window (MB)' is 10. At the bottom, there are 'Apply' and 'Discard changes' buttons.

Figure 9.7: Event Hub Capture configuration screen

Verifying Event Hub Capture output

Once Event Hub Capture is enabled, events are automatically written to Azure Storage based on the configured capture window (time or size). No additional job, consumer, or trigger is required. The only requirement is that events must flow into the Event Hub.

Event Hub Capture does not create files immediately after enabling it. Files are written only under the following circumstances:

- Events arrive at the Event Hub, and
- The capture window condition is met (time or size).

Navigate to the Azure Storage Account with the following steps:

1. Open the **Azure Portal**.
2. Navigate to the **Storage Account** configured during Capture setup.

3. From the left navigation, select **Data Storage | Containers**.
4. Open the container selected for Event Hub Capture
5. Inside the container, Event Hub Capture created a partitioned folder hierarchy automatically.
6. Navigate to the respective partition and confirm the presence of **.avro** file(s).

Figure 9.8 shows the captured Avro files in storage. The figure is as follows:

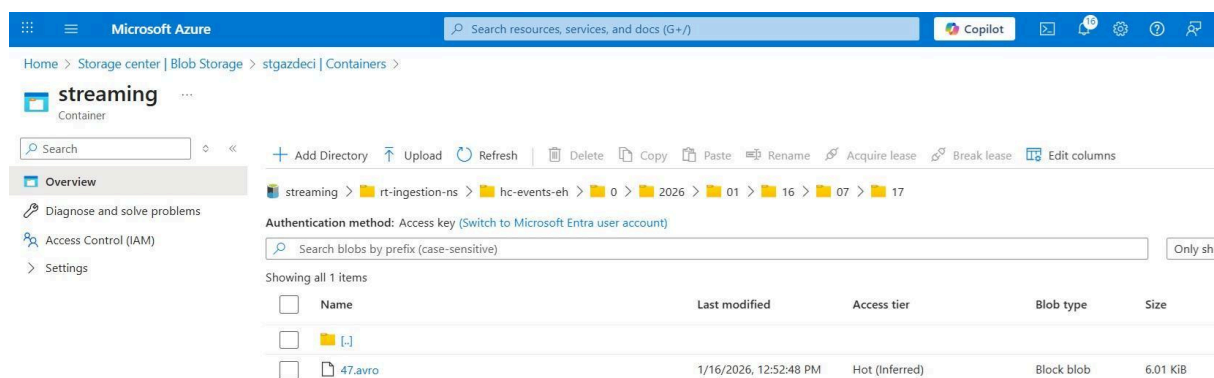


Figure 9.8: Captured Avro files in storage

Choosing the right persistence strategy

When architects first encounter Event Hub Capture, a common question arises:

Why not just consume events using Spark and write them to storage ourselves?

Both approaches are valid, but they serve very different architectural intents.

Event Hub Capture storage

Event Hub Capture is optimized for durable ingestion, not transformation. Its strengths lie in the following:

- Zero custom code.
- No always-on compute.
- Predictable, low operational cost.
- Automatic batching and partition-aware writes.

Capture is ideal under the following circumstances:

- Raw or lightly structured events must be retained.
- Re-playability is required.

- The processing logic may evolve later.
- Cost efficiency and simplicity are priorities.

In other words, Capture treats streaming data as a first-class data asset, and not just a transient signal.

Custom spark consumers

Spark Structured Streaming consumers provide the following:

- Full control over transformations.
- Stateful processing and aggregations.
- Schema enforcement and enrichment.
- Tight integration with Lakehouse architectures.

However, this power comes with the following trade-offs:

- Continuous compute costs.
- Checkpoint and offset management.
- Failure recovery complexity.
- Operational overhead.

Spark-based ingestion is appropriate in the following cases:

- Business logic must be applied immediately.
- Events are enriched or filtered before persistence.
- Streaming data feeds downstream systems directly.

Architectural guidance

A common mature pattern is to combine both approaches as follows:

- Event Hub Capture for immutable raw data persistence.
- Spark Structured Streaming for curated, business-ready datasets.

This layered strategy preserves raw truth while enabling flexible, evolving analytics.

Processing real-time data

Persisting events are only half the story. The true value of streaming systems emerges when data is processed in motion, filtered, enriched, aggregated, and prepared for downstream consumption.

Azure supports multiple streaming processing options. This section focuses on the following two processing options, with clear architectural intent:

- Spark Structured Streaming as the preferred, production-grade approach.
- Azure Stream Analytics as a lightweight, managed alternative.

Tool mastery is not the goal here; choosing the right engine for the job is.

Streaming data often represents the following:

- User behavior.
- System telemetry.
- Business events.
- IoT signals.

Processing these events as they arrive enables the following:

- Near real-time insights.
- Operational dashboards.
- Alerting and anomaly detection.
- Incremental fact generation.

Unlike batch pipelines, streaming pipelines must handle infinite data, late arrivals, and failures gracefully.

Spark Structured Streaming

Spark Structured Streaming treats streaming data as an unbounded table. Developers write declarative transformations, and Spark incrementally processes new data as it arrives. From an Azure architecture standpoint, Spark Structured Streaming is preferred because of the following factors:

- It aligns naturally with Lakehouse design.
- It shares code and patterns with batch Spark jobs.
- It scales horizontally with high throughput.
- It supports complex transformations and stateful processing.

In modern platforms such as Azure Databricks and Microsoft Fabric, Spark becomes the default processing engine for serious streaming workloads.

Typical Spark Streaming flow with Event Hubs

The following steps are the common processing pattern:

1. Events arrive in Azure Event Hubs.
2. Spark Structured Streaming reads from Event Hubs.
3. Events are parsed, validated, and enriched.
4. Aggregations or windowed calculations are applied.
5. Processed data is written to Delta tables.

This flow supports the following:

- Near real-time analytics.
- Incremental Lakehouse modeling.

Spark checkpoints ensure fault tolerance and exactly-once processing semantics when properly configured.

Azure Stream Analytics

Azure Stream Analytics (ASA) is a fully managed streaming service that emphasizes simplicity over flexibility.

It is best suited for the following:

- Simple filtering and aggregations.
- SQL-like query requirements.
- Rapid prototyping.
- Low operational overhead.

ASA integrates seamlessly with Event Hubs and downstream Azure services, but it is not designed for complex transformations or Lakehouse-centric architectures.

For this reason, ASA is often used as follows:

- An entry-level streaming processor.
- A tactical solution.
- A complement to Spark, not a replacement.

This book intentionally avoids deep ASA job authoring, as Spark offers broader applicability for data engineers.

Choosing between Spark and Stream Analytics

Azure offers multiple ways to process streaming data, and this flexibility is intentional. There is no single *best* engine for all scenarios. Instead, the right choice depends on what you want to do with the data, how complex the logic is, and how the platform will evolve.

Two services appear most often in real-world architectures: Spark Structured Streaming and Azure Stream Analytics. While both can consume data from Azure Event Hubs, they serve very different purposes.

Spark Structured Streaming should be your default choice when streaming data is part of a long-term analytical platform, not just a short-lived operational workflow.

Choose Spark Structured Streaming when the following factors apply:

- **Streaming data lands in Delta tables:** If your end goal is to store streaming data in Delta Lake, either in Azure Databricks or Microsoft Fabric, Spark is the natural fit. It supports **Atomicity, Consistency, Isolation, Durability (ACID)** guarantees, schema evolution, time travel, and unified batch-and-stream processing.
- **Transformations are non-trivial:** Real-world streaming pipelines rarely stop at filtering. They often involve the following:
 - JSON parsing and schema enforcement.
 - Enrichment with reference data.
 - Deduplication.
 - Stateful aggregations
 - Windowed computations

Spark handles these patterns cleanly and consistently.

- **The architecture follows a Lakehouse model:** In Lakehouse architectures, streaming is not a special case. It is simply another ingestion mode. Spark allows you to process batch and streaming data using the same engine, **Application Programming Interfaces (APIs)**, and storage layer.
- **Long-term scalability and extensibility are required:** Spark scales horizontally, supports high throughput, and integrates naturally with **Machine Learning (ML)** pipelines, feature engineering, and downstream analytics. If the pipeline is expected to grow in complexity over time, Spark provides the necessary headroom.

ASA shines when the goal is speed, simplicity, and operational ease, rather than deep analytical processing.

Choose Azure Stream Analytics when the following factors apply:

- **Processing logic is simple and SQL-based:** ASA is ideal for the following scenarios:

- Simple filters.
- Projections.
- Aggregations over time windows.
- If the logic can be expressed clearly in SQL, ASA keeps things straightforward.
- **Time-to-value matters more than flexibility:** ASA jobs can be set up quickly with minimal infrastructure decisions. This makes them well-suited for the following:
 - Proofs of concept.
 - Tactical solutions.
 - Lightweight operational dashboards.
- **Operational simplicity is critical:** ASA is fully managed. There are no clusters to manually size, tune, or scale. This is especially useful in the following cases:
 - The team lacks Spark expertise.
 - The workload is stable and predictable.
 - The focus is on reliability rather than extensibility.

Processing streaming data with Spark Structured Streaming

The following exercise processes streaming data with Spark Structured Streaming.

Objective: This exercise demonstrates how to consume healthcare events from Azure Event Hubs using Spark Structured Streaming, apply schema parsing and basic transformations, and persist the data into a Delta table with checkpointing enabled.

The same approach works for the following:

- Azure Databricks.
- Microsoft Fabric Lakehouse (Spark runtime).

In this book, we have used Microsoft Fabric Lakehouse. This exercise will consume the hc-events-eh events generated earlier under the rt-ingestion-ns namespace.

The following are the steps involved in the exercise:

1. Define Event Hubs connection configuration.

Spark connects to Event Hubs using a connection string stored securely. The following configuration establishes a connection between Spark Structured Streaming and Azure Event Hubs:

```

from pyspark.sql.types import *
from pyspark.sql.functions import *
from pyspark.sql import SparkSession

event_hubs_connection_string = (
    Endpoint=sb://rt-ingestion-ns.servicebus.windows.net/"
    "SharedAccessKeyName=hc-consumer-listen-policy;"
    "SharedAccessKey=<<YOUR_KEY>>;"
    "EntityPath=hc-events-eh"
)

eh_conf = {
    "eventhubs.connectionString":
    spark._jvm.org.apache.spark.eventhubs.EventHubsUtils.encrypt(
        event_hubs_connection_string
    ),
    "startingPositions": ""{"offset": "-1"}""
}

startingPositions = -1 ensures Spark starts from the earliest available
events.

```

In real production systems, offsets are usually managed through checkpoints.

2. Define the streaming DataFrame from Event Hubs with the following configuration:

```

raw_stream_df = (
    spark.readStream
        .format("eventhubs")
        .options(**eh_conf)
        .load()
)

```

Event Hub messages arrive as binary payloads inside the body column.

3. Define healthcare event schema.

This schema matches the dataset used throughout the chapter. The following configuration defines the healthcare event schema:

```

healthcare_schema = StructType([
    StructField("event_id", StringType()),
    StructField("event_type", StringType()),
    StructField("event_time", StringType()), # parse later
    StructField("source_system", StringType()),
    StructField("patient_id", StringType()),
    StructField("encounter_id", StringType()),
    StructField("provider_id", StringType()),

```

```

    StructField("department", StringType()),
    StructField("encounter_type", StringType()),
    StructField("estimated_cost", IntegerType()),
    StructField("severity_level", StringType())
  ])

```

4. Parse JSON payload and apply transformations as follows:

```

parsed_stream_df = (
  raw_stream_df
    .selectExpr("CAST(body AS STRING) AS json_payload")
    .select(from_json(col("json_payload"),
      ArrayType(healthcare_schema).alias("events")))
    .selectExpr("explode(events) as event")
    .select("event.*")
    .withColumn("event_time", to_timestamp("event_time"))
    .withColumn("ingestion_time", current_timestamp())
)

```

The following changes occur at this stage:

- JSON is deserialized.
 - Columns are strongly typed.
 - Ingestion time is added for auditability.
5. Write stream to the delta table with checkpointing as follows:

```

query = (
  parsed_stream_df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "Files/checkpoints/hc_events")
    .trigger(processingTime="30 seconds")
    .toTable("hc_events")
)

```

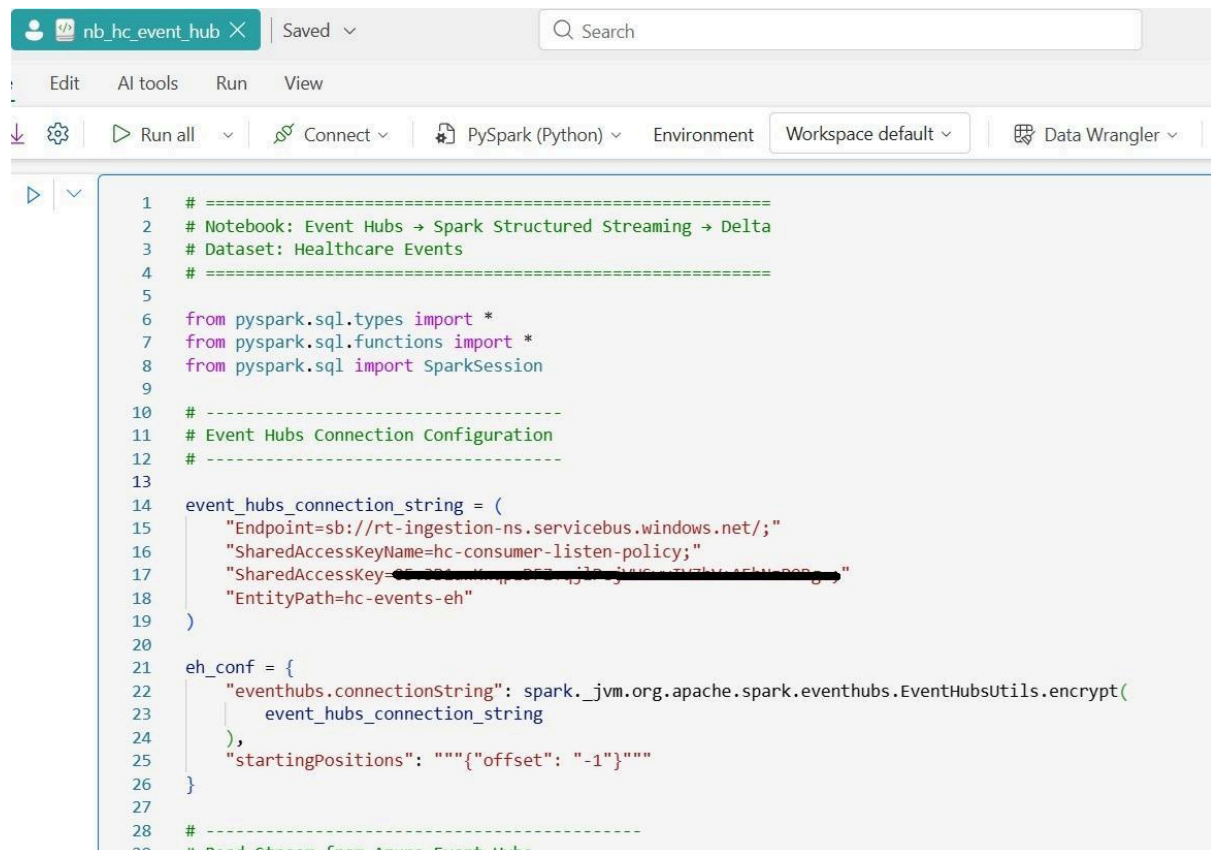
Checkpointing ensures exactly-once processing semantics and allows the stream to recover automatically after restarts or failures.

6. Display the data from the delta table with the following configuration:
- ```
display(spark.read.table("hc_events"))
```

Once started, the following actions occur:

- Spark continuously polls Event Hubs.

- New events are processed incrementally.
- Data is appended to the Delta table.



**Outcome:** Streaming events are transformed in near real time and stored in a Delta table, ready for analytics, reporting, or downstream consumption.

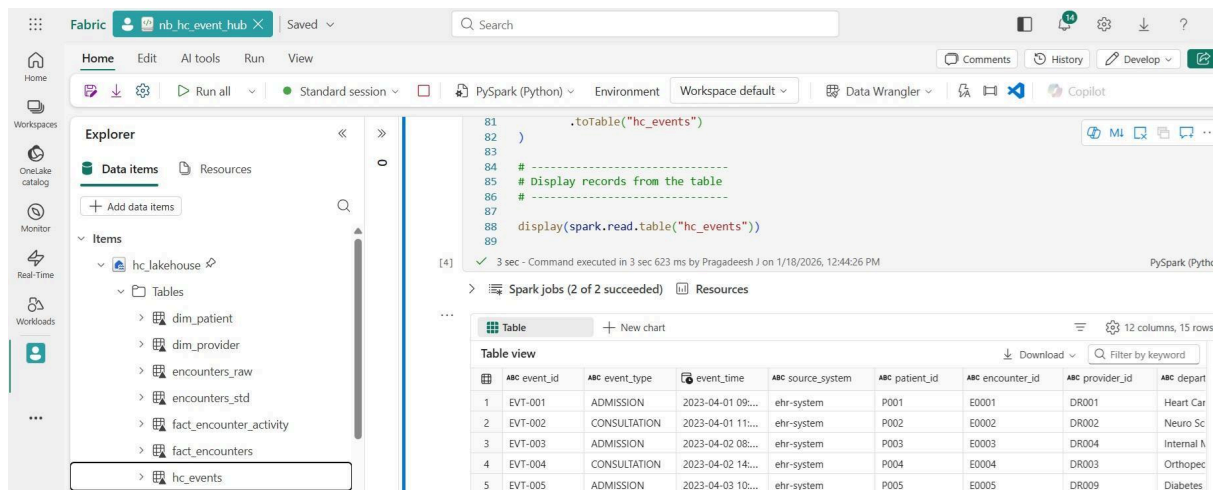


Figure 9.10: Execution of `nb_hc_event_hub` notebook in the Microsoft Fabric Lakehouse

## Integrating with other services

Azure Event Hubs is rarely used in isolation. In real-world architectures, it acts as the central ingestion layer, while downstream services consume, enrich, route, or persist events based on business needs. Azure provides native integrations that make this handoff seamless and operationally simple.

This section focuses on Azure Functions as a lightweight integration option. Unlike Spark or Stream Analytics, Azure Functions is best suited for event-driven logic, such as routing, filtering, or triggering downstream actions in response to individual events.

This approach is commonly used for alerting, notifications, lightweight transformations, or forwarding events to other systems.

## Integrating Azure Event Hubs with Azure Functions

Use Azure Functions with Event Hubs under the following cases:

- You need per-event processing, not batch analytics.
- The logic is simple and fast.
- You want a serverless, consumption-based execution model.
- Events need to be routed to APIs, queues, or operational systems.

This pattern complements Spark-based streaming rather than replacing it.

The following are the steps involved in integrating Azure Event Hubs with Azure Functions:

1. Create an Azure Function **Application (App)** with the following steps:
  - a. Open the **Azure Portal**.

- b. Navigate to **Create a resource** → **Function App**.
- c. Choose the **Consumption** option in the **Create Function App**.
- d. Give the **Function App name** as hc-events-fn and select the **Resource Group**.
- e. Choose an appropriate Runtime stack (for example, .NET).
- f. Complete the creation wizard by filling in the other necessary options.

The following figure, *Figure 9.11*, shows the configuration screen of Create Function App:

Microsoft Azure

Home > Create a resource > Create Function App >

## Create Function App (Consumption) ...

Basics Storage Networking Monitoring Deployment Authentication Tags **Review + create**

### Summary

**Function App**  
by Microsoft

**i** Basic authentication for this app is currently disabled and may impact deployments. Click to learn more.

### Details

|                                |                                      |
|--------------------------------|--------------------------------------|
| Subscription                   | 0005426e-bf2b-4d4e-abd6-c57fbeb725c5 |
| Resource Group                 | rg-azde-dev-ci                       |
| Name                           | hc-events-fn                         |
| Secure unique default hostname | Enabled                              |
| Runtime stack                  | .NET 8 (LTS), isolated worker model  |

### Hosting

**Create** < Previous Next > [Download a template for automation](#)

*Figure 9.11: Create Function App configuration*

2. **Define an Event Hub Trigger function:** With the Function App already created, the next step is to add a function that consumes events from Azure Event Hubs.

Azure currently supports creating functions using the following:

- **Visual Studio (VS).**
- VS Code (recommended for learning and demos).
- Other editors or Azure **Command Line Interface (CLI)**.

When creating the function, choose the following:

- **Trigger type:** Event Hub Trigger.
- **Event Hub name:** hc-events-eh.

- **Consumer group:** Use a dedicated consumer group for Functions.
- **Connection setting:** Reference an application setting that stores the Event Hub connection string.

This registers the function as an independent consumer of the Event Hub stream.

3. **Implement event-level processing logic:** Azure Functions is optimized for fast, event-by-event execution, not heavy analytics. The following code demonstrates filtering high-severity healthcare events and reacting immediately:

```
[FunctionName("HcEventSeverityFilter")]
public static void Run(
 [EventHubTrigger(
 "hc-events-eh",
 Connection = "EventHubConnectionAppSetting"
)] string[] events,
 ILogger log)
{
 foreach (var eventData in events)
 {
 var json = JObject.Parse(eventData);
 var severity = json["severity_level"]?.ToString();
 if (severity == "HIGH")
 {
 log.LogWarning($"High severity healthcare event detected: {eventData}");
 }
 }
}
```

4. **Observe function execution:** Once events start flowing into the Event Hub, the following execution behavior occurs:
  - Each batch of incoming events triggers the function.
  - Execution happens within seconds.
  - Logs reflect filtered or processed events.

This confirms successful integration between Event Hubs and Azure Functions.

## Design guidance

Azure Functions are best suited for event-driven, operational workloads where immediacy and simplicity are more important than scale or complex analytics.

Use Azure Functions for the following cases:



- To react immediately to incoming events, such as triggering downstream actions or notifications.
- To perform lightweight filtering, enrichment, or routing of events.
- To implement operational triggers and alerts based on real-time signals.

Avoid using Azure Functions for the following:

- Large-scale aggregations or transformations.
- Stateful or windowed analytics.
- High-throughput analytical streaming workloads that require sustained parallelism.

## Common pitfalls to avoid

The following mistakes often lead to scalability, reliability, or maintainability issues:

- Using the default consumer group that is shared with analytical streaming jobs, causing contention and lag.
- Embedding heavy transformation logic within the function, increasing execution time and cost.
- Treating Azure Functions as a replacement for analytical streaming engines such as Spark Structured Streaming or Azure Stream Analytics.
- Hardcoding Event Hub connection strings or secrets instead of using managed identities or secure configuration.

## Architectural perspective

Azure Functions provide a simple and effective integration layer between Azure Event Hubs and operational workflows. While platforms like Spark and Stream Analytics are designed for large-scale, continuous analytics, Functions excel in scenarios where low latency, event-driven processing is required.

Together, these patterns complete the Event Hubs ecosystem by enabling real-time reactions alongside real-time analytics, reinforcing Event Hubs as a versatile and resilient ingestion backbone in modern Azure architectures.

# Monitoring, scaling, and reliability

Real-time streaming systems are only as good as their ability to stay reliable under load. Unlike batch pipelines, streaming ingestion never pauses. Events keep arriving, producers keep sending, and downstream systems expect consistent throughput.

In Azure Event Hubs, reliability is achieved not through custom failover logic, but through proper monitoring, capacity planning, and partition strategy. This section focuses on the operational aspects that keep streaming pipelines healthy in production.

## Monitoring streaming systems

Streaming failures are rarely binary. Instead of a job failing, teams usually see the following:

- Increasing consumer lag.
- Intermittent throttling.
- Delayed downstream processing.
- Silent data loss due to misconfiguration.

Without proactive monitoring, these issues surface after a business impact occurs. Azure Event Hubs exposes rich metrics that allow teams to detect and respond to such issues early.

## Key Event Hub metrics to monitor

Azure Event Hubs provides built-in metrics through Azure Monitor. Among them, four are especially critical for production workloads.

### ActiveConnections

ActiveConnections represent the number of concurrent client connections established with an Event Hub namespace by producers and consumers. Each TU enforces limits on the maximum number of simultaneous connections, and exceeding these limits results in connection rejections or forced disconnects. While brief connection spikes may be harmless, sustained high active connection counts indicate inefficient client connection management or under-provisioned capacity. This can lead to increased reconnect attempts, higher latency, uneven load distribution across partitions, and potential ingestion or consumption delays, making Active Connections a critical signal for connection tuning and capacity planning.

### Incoming Messages

Incoming Messages measure the rate at which events are ingested by the Event Hub. Monitoring this metric provides visibility into producer-side load, confirms whether traffic volumes follow anticipated patterns, and offers early signals that additional throughput or scaling adjustments may be required. A sudden spike often means the following:

- A new data source was added.
- A producer is retrying aggressively.
- An upstream system is malfunctioning.

## Throttled requests

Throttling occurs when Event Hub capacity limits are exceeded, and the service restricts incoming traffic to protect system stability. While throttling is not a failure, it signals that provisioned TUs are insufficient for the current load. Frequent throttling leads to producer retries, delayed event ingestion, and can trigger cascading failures in upstream systems, making it a strong indicator that scaling or capacity adjustments are required.

## Consumer lag

Consumer lag represents the gap between the latest available events in Event Hub and the point up to which consumers have processed them. This metric is a direct indicator of downstream processing health, as high consumer lag means data has been ingested but is not yet usable for analytics or applications. It is especially critical for near real-time use cases and should be monitored at the consumer group level rather than only at the Event Hub level to accurately identify bottlenecks.

## Scaling Event Hubs using throughput units

Azure Event Hubs uses TUs to control capacity, with each TU providing defined ingress and egress throughput along with the necessary broker resources. Scaling is straightforward, as increasing TUs allows the service to handle higher event volumes, while scaling down during low-traffic periods helps optimize costs. Unlike Spark clusters, Event Hub scaling does not require redeployment or downtime. As a best practice, throttling trends should be monitored over time before increasing TUs, rather than reacting to isolated traffic spikes.

## Partition strategy and common mistakes

Partitions are a foundational concept in Azure Event Hubs. They are central to how the service achieves scale and performance, yet they are often misunderstood or treated as purely configuration-time settings. In practice, partitioning decisions have long-term implications on throughput, consumer behavior, and operational flexibility.

## Role of partitions in Event Hubs

Partitions serve the following three critical purposes:

- **Enable parallel ingestion and consumption:** Events written to an Event Hub are distributed across multiple partitions, allowing multiple producers and consumers to

operate in parallel. This parallelism is what enables Event Hubs to handle high event volumes without becoming a bottleneck.

- **Preserve event ordering within a partition:** While Event Hubs does not guarantee global ordering, it does guarantee that events within a single partition are delivered in the order in which they were received. This makes partitions the natural boundary for ordering-sensitive workloads.
- **Distribute load across the service infrastructure:** Each partition is backed by independent storage and processing resources. By spreading events across partitions, Event Hubs balances load across brokers, improving reliability and preventing hot spots under sustained or bursty traffic.

Together, these characteristics make partitions the primary scaling unit in Event Hubs, one that must be sized and used intentionally to support both current and future streaming workloads.

## Common partitioning strategy pitfalls

The common partitioning strategy pitfalls are as follows:

- **Under-partitioning (Too few partitions):** Choosing too few partitions restricts parallel consumption and limits the ability of consumer groups to scale. As event volume increases, this often results in growing consumer lag and slower processing. Since partitions cannot be reduced once created, early under-partitioning becomes difficult and costly to correct later.
- **Over-partitioning (Too many partitions):** Creating an excessive number of partitions introduces unnecessary overhead across producers, consumers, and downstream processing systems. Over-partitioning can also lead to inefficient resource utilization, uneven data distribution, and increased operational complexity without delivering proportional performance benefits.
- **Poor or missing partition key design:** Ignoring partition keys or choosing poorly distributed keys causes events to concentrate on a small subset of partitions, creating hot partitions. This results in uneven load distribution, increased latency, and throttling scenarios even when sufficient TUs are provisioned.

## Rule of thumb for partition design

Always design partitions based on expected consumer parallelism, rather than producer count alone. While producers can scale independently, partitions ultimately define the maximum parallelism available to consumer groups and therefore determine downstream processing capacity.

# Reliability patterns for Event Hub pipelines

Reliability in Event Hubs is primarily achieved through **architectural design decisions**, not through aggressive or custom retry logic. Well-designed pipelines naturally absorb traffic variability and fail gracefully under load.

Key reliability practices include the following:

- Isolating workloads using separate consumer groups, ensuring that independent processing paths do not interfere with one another.
- Enabling Event Hub Capture to persist raw events to durable storage, providing replay and recovery capabilities.
- Monitoring consumer lag, rather than relying solely on throughput metrics, to detect downstream bottlenecks early.
- Scaling TUs proactively, before throttling transitions from transient to sustained behavior.

When applied together, these practices allow Event Hubs to function as a shock absorber between producers and consumers, smoothing traffic spikes while protecting downstream systems from overload.

## Monitoring and scaling Azure Event Hubs

The following exercise consolidates monitoring and scaling into a single operational workflow.

**Objective:** Monitor Event Hub health and scale capacity to maintain reliable streaming ingestion.

The following are the steps involved:

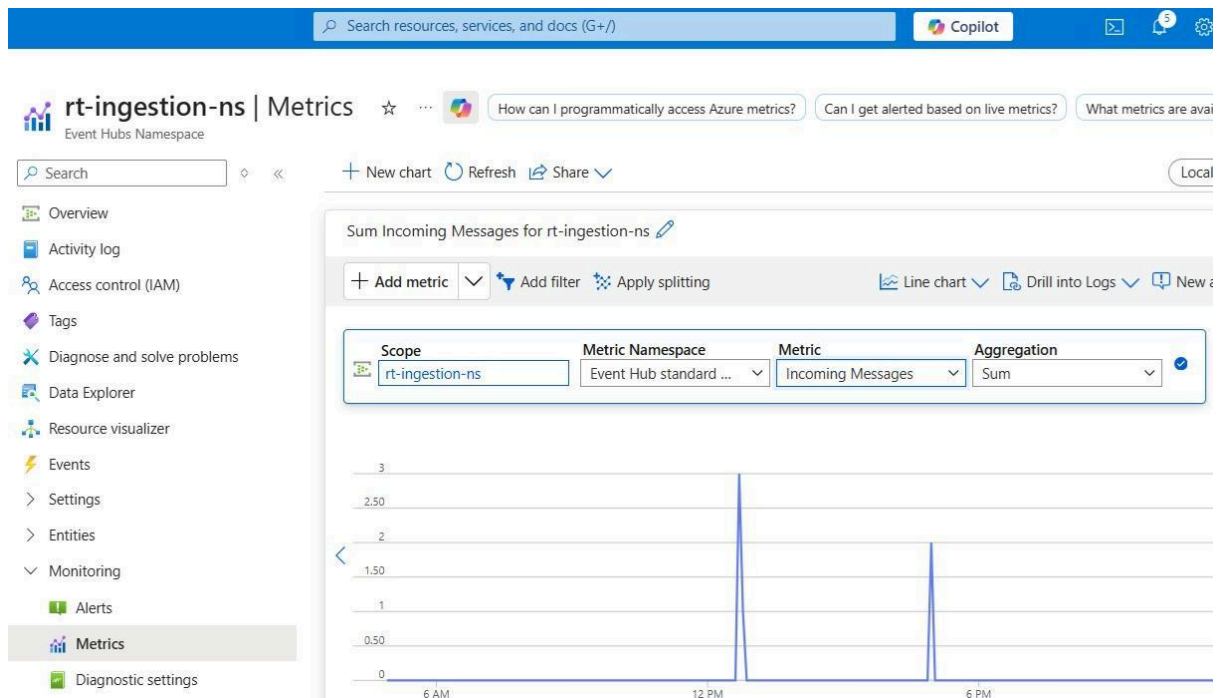
1. Open the **Azure Portal** and navigate to your Event Hubs Namespace (rt-ingestion-ns).
2. In the **namespace blade**, scroll down the left menu and look for Monitoring.
3. Select **Metrics** under the Monitoring section.
4. Click + **New chart**
5. Click + **Add metric**
6. Select values from the drop-downs as follows:
  - **Scope** | **rt-ingestion-ns**
  - **Metric Namespace** | **Event Hub Standard Metrics**
  - **Metric** | **Incoming Messages**

- **Aggregation | Sum =**

7. Repeat the same configuration for the **ActiveConnections** metric, but set the Aggregation to Max instead of Sum.

**Outcome:** After configuring the charts, the Metrics dashboard displays time-series views of Incoming Messages and Active Connections. Incoming Messages indicates the ingestion rate, while Active Connections reflects concurrent producer and consumer connections. Under normal conditions, both remain within the limits defined by the provisioned throughput units. Sustained increases in Active Connections signal rising concurrency or inefficient connection management, helping validate whether the Event Hub is operating within its intended capacity.

The following figure, *Figure 9.12*, shows the Incoming Messages using a line chart:



*Figure 9.12: Incoming Messages metric over time*

The following figure, *Figure 9.13*, shows Active Connections using a line chart

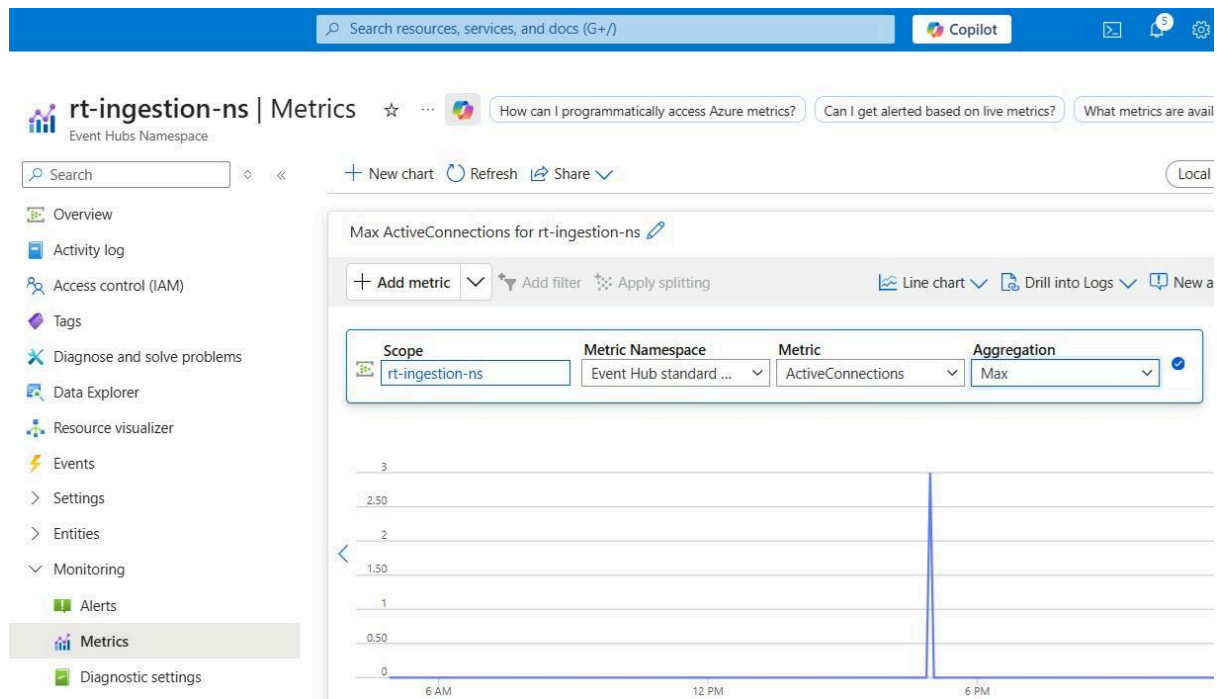


Figure 9.13: ActiveConnections metric over time

Monitoring and scaling are not optional tasks in streaming architectures. They are continuous responsibilities. Teams that treat Event Hubs as a *set-and-forget* service often discover issues too late. Teams that monitor intentionally build trustworthy, production-grade streaming platforms.

## Securing Event Hub access and managing permissions

Security in streaming platforms is often underestimated because ingestion systems are designed to be highly available and loosely coupled. However, Azure Event Hubs frequently acts as the front door of the enterprise data platform, ingesting business events, telemetry, and customer signals. A weak security posture at this layer exposes the entire downstream analytics ecosystem.

This section focuses on production-grade security decisions for Azure Event Hubs, covering authentication models, network isolation, authorization design, and common anti-patterns observed in real-world deployments.

## Authentication models in Azure Event Hubs

Azure Event Hubs supports the following two primary authentication approaches:

- **Shared Access Signatures (SAS).**
- **Managed Identity (Azure Active Directory (AD)-based).**

Choosing the right model has long-term implications for governance, rotation, and operational safety.

## Shared Access Signatures

SAS is a token-based authentication mechanism using shared keys. Access is granted through Shared Access Policies, which define permissions as follows:

- Send.
- Listen.
- Manage.

SAS is more appropriate in the following scenarios:

- Proof-of-concept or sandbox environments.
- External producers without Azure AD integration.
- Simple integration scenarios.

The following are its limitations:

- Secrets must be stored and rotated.
- Keys are often hardcoded or poorly managed.
- Difficult to audit at scale.

SAS should be treated as a controlled exception, not the default security model for enterprise streaming platforms.

## Managed Identity

Managed Identity uses Azure AD to authenticate services without secrets.

The following are its key advantages:

- No credentials stored in code or configuration.
- Centralized access control using Azure **Role-Based Access Control (RBAC)**.
- Automatic token rotation.
- Strong auditability.



The following are its common usages:

- Azure Databricks.
- Azure Functions.
- Azure Stream Analytics.
- Azure Data Explorer.

For production systems, Managed Identity is the preferred and recommended approach.

## Authorization and least privilege

A common security failure in streaming systems is granting overly broad permissions *just to make it work*. Since streaming platforms scale rapidly, a poor permission design becomes difficult to unwind later.

The following are the best practices:

- Separate **Send** and **Listen** permissions.
- Avoid using the **Manage** permission for applications.
- Use distinct identities for producers and consumers.
- Scope access at the Event Hub level where possible.

Using Azure RBAC roles such as Event Hubs Data Sender and Event Hubs Data Receiver ensures permissions remain explicit, minimal, and auditable.

## Network access and isolation

Security is not only about authentication; it also includes network boundaries.

Azure Event Hubs supports the following **Network Access Controls (NAC)**:

- Public endpoint access.
- **Internet Protocol (IP)** filtering.
- Private endpoints.

## Private endpoints best practice

Private endpoints allow Event Hubs to be accessed only from within a virtual network.

The following are its benefits:

- Traffic does not traverse the public internet.
- Reduced attack surface.

- Strong compliance alignment.

For regulated or mission-critical workloads, Event Hubs should be exposed only through private endpoints.

## Common security anti-patterns

The following anti-patterns appear frequently in real-world systems:

- **Anti-pattern 1:** Shared SAS keys across teams.

It leads to the following:

- No ownership clarity.
- High blast radius.
- Unsafe key rotation.

- **Anti-pattern 2:** Long-lived SAS tokens.

It creates the following:

- Persistent exposure.
- Compliance issues.
- Delayed revocation.

- **Anti-pattern 3:** Public network access for Production Hubs.

It increases the following:

- Attack surface.
- Accidental exposure risk.
- Compliance violations.

- **Anti-pattern 4:** Reusing consumer groups across systems.

It leads to the following:

- Unpredictable lag.
- Operational confusion.
- Debugging complexity.

## Design takeaway

Security in streaming architectures is not a late-stage configuration or a checklist item to revisit after pipelines are live; it is an architectural commitment made at the very beginning. Azure Event Hubs often sits at the outermost edge of the data platform, ingesting data from applications, devices, partner systems, and external services. Any weakness at this boundary

does not remain isolated; it propagates downstream into processing engines, storage layers, analytics models, and reporting systems.

Once producers and consumers are integrated and operating at scale, security decisions become difficult and expensive to change. Connection strings, consumer groups, and network paths tend to spread across codebases, deployment pipelines, and operational runbooks. For this reason, insecure defaults such as shared secrets, overly broad permissions, or public network exposure tend to persist far longer than intended.

Azure Event Hubs provides strong primitives for building secure-by-default streaming architectures, including identity-based access through Azure AD and network isolation through private endpoints. When these capabilities are applied deliberately, they allow platforms to scale ingestion and consumption without relying on embedded secrets, manual key rotation, or implicit trust in public network boundaries.

The following architectural principles guide a disciplined security posture for Event Hubs:

- **Favor identity over secrets:** Prefer managed identities and Azure RBAC over shared access keys. Identity-based access eliminates long-lived secrets, improves auditability, and aligns Event Hubs security with modern, enterprise-wide identity governance practices.
- **Minimize permissions by default:** Grant only the permissions required for a specific role; send for producers, listen for consumers, and avoid managing access for applications. Least-privilege designs reduce blast radius and make operational behavior easier to reason about as the platform scales.
- **Isolate networks wherever possible:** Treat Event Hubs as a protected ingress point. Use private endpoints to ensure that traffic remains within trusted network boundaries and does not traverse the public internet, especially for regulated or business-critical workloads.
- **Design for auditability and long-term maintainability:** Security controls should be explicit, visible, and centrally governed. RBAC-based access, clear identity ownership, and periodic policy reviews ensure that the platform remains understandable and defensible over time.

When applied consistently, these principles elevate Azure Event Hubs from a high-throughput messaging service into a secure and trustworthy entry point for real-time data. Rather than slowing teams down, strong early security decisions simplify operations, reduce risk, and enable streaming platforms to grow confidently as business demands increase.

# Troubleshooting and operational best practices

Real-time streaming platforms often have a reputation for being fragile or unpredictable. In practice, most production issues with Azure Event Hubs are not caused by the platform itself, but by misunderstandings about how streaming systems behave.

This section addresses the most common operational questions teams face after going live, including event loss concerns, reprocessing needs, and the practical limits of replay mechanisms such as Event Hub Capture.

## The myth of event loss

One of the most frequent concerns raised by teams new to streaming systems is: *Are we losing events?*

This fear is understandable. In traditional batch systems, missing data usually means something failed visibly. In streaming systems, however, the behavior is different and often misleading at first glance.

In most real-world scenarios, events are not lost. Instead, they are delayed, skipped by consumers, or no longer available due to retention limits. The following are the practical examples:

- **Delayed, not lost:** Imagine a hospital system sending admission and consultation events into Azure Event Hubs every second. Suddenly, downstream analytics dashboards stop updating for a few minutes.

The immediate assumption is often: *Event Hub dropped our data.*

In reality, the following typically occurs:

- Consumer applications (Spark streaming, Stream Analytics, Functions) fell behind.
- The consumer lag increased due to compute pressure or scaling limits.
- Events accumulated safely inside Event Hub partitions.

Once the consumers recover or scale out, they resume reading from the last committed offset, and the *missing* data suddenly appears. The events were delayed, not lost.

- **Skipped events due to consumer group behavior:** Consider the following two teams consuming the same healthcare events:
  - One Spark job writing to a Delta Lake.
  - One Azure Function triggering alerts for critical cases.

If both applications accidentally use the same consumer group, they begin competing for events. Event Hubs delivers each event once per consumer group, not once per consumer.

The following occurs:

- Some events go to Spark.
- Some go to the Function.
- Neither consumer sees the full dataset.

From each team's perspective, the data was lost, but the events were actually delivered exactly as designed.

- **Retention window expiry:** It is the most common misunderstanding. Event Hubs is not a permanent storage system. If event retention is configured for a day and a consumer is down for 36 hours, older events are automatically deleted.

When the consumer restarts, the following situations may occur:

- It attempts to read from its last known offset.
- That offset no longer exists.
- The consumer resumes from the earliest available event instead.

At this point, historical events are genuinely unavailable, but they were not lost unexpectedly. They expired exactly as configured.

**Sending events before the consumer started:** It is a very common learning-phase issue, as follows:

- Events are sent to Event Hub
- A streaming job is started later
- No data appears in the output

By default, many consumers start reading from the latest offsets instead of the beginning of the stream. This makes it seem like earlier events vanished.

Unless explicitly configured to read from the earliest available offset, the consumer will only see new events arriving after it starts.

## Mental model shift

In streaming systems, the following principles apply:

- Event Hubs guarantees delivery within retention.
- Consumers are responsible for tracking progress.
- Offsets, not files, define data position.

What feels like *event loss* is almost always the following:

- A consumer lag issue.
- A configuration mismatch.
- A retention expectation gap.

True data loss is rare and usually tied to misconfigured retention or deliberate cleanup, not platform instability.

## Diagnosing streaming issues systematically

When issues arise, avoid guessing. A structured approach works best as follows:

- Check Incoming Messages to confirm producers are sending data.
- Inspect Throttled Requests to detect capacity pressure.
- Review Consumer Lag per consumer group.
- Verify retention settings against processing delays.
- Confirm partition key usage and distribution.

In practice, one of these checks almost always explains the symptom observed downstream.

## Reprocessing strategies in Event Hub architectures

In mature streaming architectures, reprocessing is not an exception but an expectation. Business logic evolves, bugs surface, and downstream systems may require corrections. Azure Event Hubs provides limited but effective reprocessing capabilities, and successful implementations deliberately combine multiple techniques to handle replay scenarios safely.

### Reprocessing using consumer groups

Creating a new consumer group is the simplest way to re-read events from Azure Event Hubs without disrupting existing workloads. Each consumer group maintains its own independent offset. This means that a new consumer group starts reading events as if it were the first time, while other consumers continue unaffected.

This approach works well in the following scenarios:

- A bug is discovered in downstream processing.
- A transformation logic needs to be corrected.
- A new analytical consumer needs recent historical data.

If the events are still available in Event Hub retention, a new consumer group can be created, but please note that the following are its limitations:

- While consumer-group-based reprocessing is convenient, it operates within strict platform boundaries.
- **Retention is time-based, not volume-based.** Event Hubs retains data for a fixed duration (for example, one day, seven days, or up to 90 days, depending on the tier). Once this time window expires, events are permanently removed regardless of whether they were consumed or not.

This means the following:

- You cannot reprocess events older than the retention window.
- Increasing traffic does not extend retention.
- Missed events cannot be recovered once expired.
- **Reprocessing only works while data is still retained.** If processing is delayed due to downstream outages, throttling, or consumer lag, reprocessing may no longer be possible by the time the issue is discovered.

For example, A Spark streaming job is down for two days, but Event Hub retention is set to one day. When the job restarts, the older events are already gone, and no consumer group can recover them.

When it comes to design guidance, consumer groups are best suited for the following:

- Short-term replays.
- Debugging and validation.
- Non-critical historical corrections.

They should *not* be treated as a long-term recovery mechanism.

## Reprocessing using Event Hub Capture

When Event Hub Capture is enabled, every incoming event is automatically written to Azure storage (Data Lake or Blob) in near real time. Instead of existing only as transient messages inside Event Hubs, events become durable files that can be replayed long after the retention window has expired.

This fundamentally changes how streaming data is treated.

Capture enables the following:

- Full historical replay of raw events.
- Schema evolution handling, since raw payloads are preserved.

- Batch reprocessing using Spark, SQL, or other analytics engines.

In effect, Capture converts a streaming ingress into a long-term data asset, making downstream corrections and reprocessing both practical and predictable.

While Event Hub Capture is powerful, it is important to understand what it is *not*. Capture is not a streaming engine and does not behave like a real-time consumer.

The key limitations include the following:

- Capture files are append-only; updates and deletes are not supported.
- Event ordering is preserved only within a partition, not globally.
- Replay is batch-oriented, not event-by-event.
- There is no built-in offset or checkpoint management for replay consumers.

This means replaying data from Capture requires external logic, typically Spark, Fabric, or SQL, to manage progress, deduplication, and idempotency.

When it comes to design guidance, Capture works best in the following cases:

- Data is business-critical and must be replayable.
- Retention windows alone are insufficient.
- Streaming feeds are part of a Lakehouse architecture.
- Downstream systems may need correction or re-computation.

Capture should be paired with the following:

- Spark Structured Streaming for analytics.
- Batch reprocessing jobs for backfills.
- Delta tables for versioned, auditable datasets.

## Operational best practices

When streaming systems move from diagrams to production, a few rules quickly separate stable pipelines from fragile ones. Teams that operate Event Hubs successfully tend to follow the following practices, often learned through real incidents:

- **Treat Event Hubs as stateless ingestion, not business logic:** Use Event Hubs to receive and distribute events reliably. Apply validation, enrichment, and business rules downstream where changes are easier and safer.



- **Never rely on a single consumer group for multiple workloads:** Give each workload its own consumer group. This prevents slow consumers from blocking fast ones and makes failures easier to isolate.
- **Enable Capture for critical data streams:** If the data matters, keep a copy. Capture provides a durable fallback when downstream processing needs to be corrected or replayed.
- **Monitor lag, not just throughput:** High ingress rates can hide problems. Consumer lag is the earliest and most reliable signal that something is falling behind.
- **Document retention and replay expectations clearly:** Make it explicit how long data can be reprocessed. During incidents, this clarity avoids panic and unrealistic recovery assumptions.

These rules are simple, but they are rarely obvious at the beginning. Most teams adopt them after experiencing missed events, stalled consumers, or painful reprocessing efforts.

## Building operational confidence

Operational excellence in streaming systems is not about memorizing features or tuning every knob. It comes from understanding how the system behaves under load, how it fails, and where its boundaries are.

Teams that diagnose issues methodically, choose reprocessing strategies with intent, and respect platform constraints build pipelines that are not only resilient but also predictable. Problems become manageable, recoveries become routine, and production incidents lose their mystery.

When designed and operated with these principles in mind, Azure Event Hubs becomes a reliable ingestion backbone supporting both real-time processing and long-term analytical needs with confidence, rather than surprises.

## Conclusion

In this chapter, you explored Azure Event Hubs as a foundational service for building scalable, reliable, and cost-efficient real-time streaming ingestion platforms on Azure. You learned how Event Hubs acts as a high-throughput, low-latency ingestion layer that is capable of absorbing events from diverse sources while decoupling producers from downstream consumers.

Through a layered architectural approach, the chapter explained how streaming data flows from event sources into Event Hubs, how it can be captured durably using Event Hub Capture, and how different processing engines such as Spark Structured Streaming, Azure Stream Analytics, and Azure Data Explorer can be applied based on workload complexity and business requirements. This progression highlighted the importance of separating ingestion, persistence, and processing concerns in modern streaming architectures.

The chapter also focused on practical, hands-on experience in setting up Event Hub namespaces, configuring partitions, managing access using Shared Access Policies and Managed Identity, and operating Event Hubs in a cost-conscious manner. The cookbook-style recipes demonstrated how to monitor streaming workloads, scale throughput units, and design partition strategies that avoid common operational pitfalls.

Beyond implementation, the chapter addressed real-world operational realities. You examined common misconceptions around event loss, learned effective reprocessing strategies, and understood the practical limitations of replay mechanisms such as Event Hub Capture. These insights help teams move from experimental streaming setups to production-grade, trustworthy ingestion platforms.

By the end of this chapter, you should be able to confidently design and operate real-time ingestion pipelines that balance throughput, reliability, security, and cost. More importantly, you now understand how Azure Event Hubs fits into a broader analytics ecosystem, serving as the entry point that enables downstream Lakehouse, real-time analytics, and reporting solutions.

With these foundations in place, you are well prepared to integrate streaming ingestion into modern data platforms, ensuring that real-time data becomes a dependable and actionable asset rather than an operational challenge.

 In the next *Chapter 10, Unified Data Management and Analytics Using Microsoft Data Fabric*, you will learn how to manage and analyze data using **Microsoft Data Fabric**, which includes setting up Data Fabric, integrating data sources, and performing unified analytics. **Microsoft Data Fabric** provides a holistic approach to data management and analytics.